

BÀI TẬP TÌM KIẾM TUYẾN TÍNH

(Linear Search) • 20 bài tập theo 3 mức độ tăng dần

Thuật toán **tìm kiếm tuyến tính** duyệt lần lượt từng phần tử của dãy, so sánh với giá trị cần tìm **x** cho tới khi tìm thấy hoặc duyệt hết dãy. Bộ bài tập dưới đây giúp bạn nắm vững từ ý tưởng thuật toán đến cài đặt và vận dụng. Có thể lập trình bằng Python (như ví dụ trong bài học) hoặc ngôn ngữ tùy chọn. **Quy ước:** mảng đánh chỉ số từ 0; hàm trả về **-1** khi không tìm thấy.

PHẦN A — HIỂU THUẬT TOÁN (Mức cơ bản)

1 Bài 1. Trình bày ý tưởng

Bằng lời của bạn, hãy mô tả ý tưởng của thuật toán tìm kiếm tuyến tính. Nêu rõ **Input** và **Output** của thuật toán, và cho biết thuật toán dừng lại trong những trường hợp nào.

2 Bài 2. Mô phỏng từng bước

Cho mảng $A = [7, 3, 9, 12, 5, 8, 1]$ và $x = 5$. Hãy lập bảng mô phỏng từng bước: ở mỗi bước ghi rõ chỉ số i , giá trị $A[i]$, kết quả so sánh $A[i]$ với x , và kết luận. Cuối cùng cho biết giá trị mà hàm trả về.

3 Bài 3. Đếm số phép so sánh

Vẫn với mảng A ở Bài 2, hãy đếm số phép so sánh ($A[i] == x$) cần thực hiện để tìm: (a) $x = 7$; (b) $x = 1$; (c) $x = 100$ (không có trong mảng). Em rút ra nhận xét gì về vị trí của phần tử và số phép so sánh?

4 Bài 4. Phân tích độ phức tạp

Một mảng có n phần tử. Hãy cho biết số phép so sánh trong: (a) trường hợp tốt nhất; (b) trường hợp xấu nhất; (c) trung bình (khi phần tử có trong mảng). Từ đó suy ra độ phức tạp thời gian của thuật toán theo ký hiệu O lớn.

5 Bài 5. Điều kiện áp dụng

Tìm kiếm tuyến tính có bắt buộc mảng phải được sắp xếp trước hay không? Giải thích. So sánh ngắn gọn với **tìm kiếm nhị phân** về: điều kiện áp dụng và độ phức tạp.

PHẦN B — LẬP TRÌNH CƠ BẢN (Mức trung bình)

6 Bài 6. Hàm tìm kiếm cơ bản

Viết hàm `linear_search(a, x)` trả về **vị trí đầu tiên** của x trong mảng a , trả về **-1** nếu không có. Viết chương trình nhập mảng, nhập x và in kết quả ra màn hình.

```
def linear_search(a, x):  
    # ... viết phần thân hàm ở đây  
    return -1
```

7 Bài 7. Kiểm tra tồn tại

Viết hàm `ton_tai(a, x)` trả về **True** nếu `x` có mặt trong mảng, ngược lại trả về **False** (không dùng toán tử `in` có sẵn).

8 Bài 8. Đếm số lần xuất hiện

Viết hàm `dem_xuat_hien(a, x)` đếm số lần `x` xuất hiện trong mảng.

Ví dụ: `a = [2, 5, 2, 7, 2]`, `x = 2` → kết quả 3.

9 Bài 9. Tìm tất cả vị trí

Viết hàm `tim_tat_ca(a, x)` trả về danh sách **tất cả** các chỉ số mà `x` xuất hiện. Nếu không có, trả về danh sách rỗng.

Ví dụ: `a = [4, 1, 4, 9, 4]`, `x = 4` → `[0, 2, 4]`.

10 Bài 10. Vị trí xuất hiện cuối cùng

Viết hàm trả về **vị trí xuất hiện cuối cùng** của `x` trong mảng (trả về -1 nếu không có). Gợi ý: duyệt từ cuối mảng về đầu, hoặc cập nhật biến lưu vị trí khi duyệt từ đầu.

11 Bài 11. Tìm giá trị lớn nhất

Không dùng hàm `max()` có sẵn, hãy dùng kỹ thuật duyệt tuyến tính để tìm **giá trị lớn nhất** và **vị trí** của nó trong mảng.

12 Bài 12. Min và Max trong một lần duyệt

Chỉ với **một lần duyệt** mảng, hãy tìm đồng thời giá trị nhỏ nhất và giá trị lớn nhất, đồng thời in ra vị trí tương ứng của chúng.

PHẦN C — VẬN DỤNG & NÂNG CAO (Mức nâng cao)

13 Bài 13. Tìm kiếm trên chuỗi

Cho danh sách tên sinh viên (kiểu chuỗi). Viết hàm tìm xem một tên có trong danh sách không, **không phân biệt chữ hoa/thường**, trả về vị trí tìm thấy hoặc -1.

Ví dụ: `ds = ["An", "Bình", "Châu"]`, tìm `"an"` → vị trí 0.

14 Bài 14. Tìm theo điều kiện

Viết hàm tìm **số chẵn đầu tiên** trong mảng và trả về vị trí của nó. Nếu mảng không có số chẵn nào, trả về -1.

Ví dụ: `a = [3, 7, 11, 8, 5, 4]` → số chẵn đầu tiên là 8 tại vị trí 3.

15 Bài 15. Tìm số nguyên tố đầu tiên

Cho mảng số nguyên dương. Viết hàm tìm **số nguyên tố đầu tiên** trong mảng, trả về giá trị và vị trí của nó. Gợi ý: viết thêm hàm phụ kiểm tra số nguyên tố.

Ví dụ: `a = [4, 6, 9, 7, 11]` → số nguyên tố đầu tiên là 7 tại vị trí 3.

16 Bài 16. Phần tử gần x nhất

Cho mảng số và giá trị x . Viết hàm tìm phần tử trong mảng có giá trị **gần x nhất** (chênh lệch tuyệt đối nhỏ nhất). Trả về giá trị và vị trí của phần tử đó.

Ví dụ: $a = [10, 22, 28, 29, 40]$, $x = 26 \rightarrow$ gần nhất là 28 tại vị trí 2.

17 Bài 17. Tìm kiếm có lính canh (Sentinel)

Cài đặt tìm kiếm tuyến tính theo kỹ thuật **lính canh**: tạm đặt giá trị x vào cuối mảng làm "lính canh", nhờ đó vòng lặp chắc chắn dừng và **bỏ bớt** một điều kiện kiểm tra biên ($i < n$) trong mỗi vòng. Hãy so sánh số phép so sánh với cách thông thường.

18 Bài 18. Tìm kiếm trên ma trận 2 chiều

Cho ma trận (mảng 2 chiều) các số nguyên. Viết hàm tìm kiếm tuyến tính giá trị x trong ma trận, trả về vị trí (**dòng, cột**) đầu tiên tìm thấy, hoặc $(-1, -1)$ nếu không có.

Ví dụ: $M = [[5, 8, 1], [3, 9, 7], [2, 6, 4]]$, $x = 9 \rightarrow (1, 1)$.

19 Bài 19. Tìm kiếm theo khóa (dữ liệu có cấu trúc)

Cho danh sách sinh viên, mỗi sinh viên gồm: **mã SV, họ tên, điểm trung bình** (dùng dictionary hoặc class). Viết hàm tìm sinh viên theo **mã SV** và in đầy đủ thông tin của sinh viên đó; thông báo phù hợp nếu không tìm thấy.

20 Bài 20. Bài toán tổng hợp — Quản lý danh bạ

Viết chương trình quản lý danh bạ điện thoại đơn giản, sử dụng tìm kiếm tuyến tính cho các chức năng: (1) thêm liên hệ gồm *tên* và *số điện thoại*; (2) tìm số điện thoại theo *tên*; (3) tìm *tên* theo số điện thoại; (4) đếm số liên hệ có số điện thoại bắt đầu bằng một đầu số cho trước (ví dụ "090"). Thiết kế menu lặp lại cho người dùng chọn chức năng.

Gợi ý ôn tập: với mỗi bài lập trình, hãy thử kiểm tra thêm các trường hợp đặc biệt như mảng rỗng, phần tử nằm ở đầu/cuối mảng, và phần tử không tồn tại. Chúc bạn học tốt!